Kaggle Capstone Project

Project Title: Predicting Company Sales Across Quarters Challenge

Type: Individual

Author: Shravan Kumaar Venkatesh Kumar
NetID:GA8997

1. Introduction

Overview of Business Problem

Sales forecasting is one of the most critical tasks for companies across all industries. Reliable sales predictions allow organizations to:

- Optimize inventory management (avoiding overstocking and stockouts),
- Align marketing and promotional efforts with expected demand,
- Manage cash flows and budgeting based on revenue forecasts,
- Plan operational capacities (labor, logistics, supply chain needs), and
- Strengthen strategic planning for future growth.

Inaccurate sales forecasts can lead to substantial financial risks including:

- Tied-up capital due to excess inventory,
- Lost revenue due to stock shortages,
- Poor investment decisions,
- Customer dissatisfaction,
- Workforce mismanagement.

Thus, there is a strong business need for a robust, reliable, and accurate predictive model that can estimate future quarterly sales based on both internal company factors and external economic conditions.

Specific Business Context for this Challenge:

- **Dataset:** Includes historical company-level financial indicators and regional/national economic metrics.
- **Goal:** Predict quarterly sales for different companies based on these features.
- **Business Use:** A high-accuracy predictive model would allow companies to proactively adjust operations quarter-by-quarter, reducing waste, increasing profitability, and gaining competitive advantage.

Purpose of the Predictive Model

The predictive model would be used to:

- Generate automated quarterly forecasts,

- Support executive decision-making (finance, operations, marketing),
- Drive inventory optimization systems,
- Identify high-risk companies or regions for targeted interventions.

Example:

If a model predicts that a company's sales will decline sharply next quarter, management can lower production targets and reduce marketing budgets accordingly.

Choice of Evaluation Metric

The chosen evaluation metric is Mean Absolute Error (MAE):

- MAE directly measures the average magnitude of prediction errors, without considering their direction.
- It is interpretable in business units (i.e., how many dollars on average the prediction is wrong by).
- It treats over-predictions and under-predictions equally, avoiding bias in sales forecasting scenarios.

Production-Ready Benchmark for Model Deployment

To be adopted into a real-world production environment, the predictive model should achieve:

- Validation MAE below 600 units,
- Kaggle leaderboard MAE competitive to top 20% of participants, ensuring strong generalization.

Business Acceptable Error Threshold:

An error rate below 10% of typical quarterly sales would be considered acceptable for operational deployment.

Given that typical quarterly sales in the dataset range around $5,000–$15,000, achieving an MAE below $600 meets that operational threshold (~5-10% error).

Overview of Best Solution

The best performing model was:

- Model Type: LightGBM (Gradient Boosted Decision Trees, boosting type=gbdt)
- Tuning: Optuna hyperparameter optimization
- Features: Full feature set with additional engineered interaction features

- Encoding: Target Encoding for categorical features (Company, Region, Industry, etc.)
- Validation Strategy: Stratified random splits with StandardScaler applied

Performance Metrics:

- Validation MAE: ~646
- Kaggle Public Leaderboard MAE: 537.85

This solution proved to be stable, generalizable, and interpretable, fulfilling the business requirements for model deployment.

Summary of Report Structure

This report is organized as follows:

1. Data Section: Description of the features, relationship to sales, exploratory data analysis (EDA) and preprocessing steps.
2. Methods Section: Modeling approaches tested, feature engineering experiments, and tuning strategies.
3. Results Section: Documentation of each submission, detailed error analysis, ranking across submissions, and learning reflections.
4. Analysis Section: Lessons learned and key insights from the modeling process.
5. AI Contributions: How GenAI tools were used to accelerate development.
6. Reflection on the Future of Analytics and GenAI: Perspective on evolving analyst roles.
7. Project Execution Plan: Detailed timeline and task plan for model deployment in a business setting.

2. Data

2.1 Dataset Overview

The dataset provided for the Kaggle Challenge consists of structured tabular data where:

- Each row represents a single observation for a company in a given quarter.
- The dependent variable to predict is Sales — a continuous, positive variable representing company revenue.

The data can be broadly classified into two categories:

1. Company-specific features: Financial health indicators, growth metrics, regional operations, and ratings.
2. Macroeconomic features: External economic indicators representing broader market conditions.

2.2 Features and Their Meaning

Feature	Description	Expected Impact on Sales
Quarter	Fiscal quarter of the observation (1–4)	Seasonality and holiday effects
Company	Unique identifier for each company	Captures company-specific growth patterns
Region	Geographic region (East, West, North, South)	Regional economic health differences
Industry	Industry sector classification	Some industries are more seasonal or recession-proof
Bond Rating	Company's creditworthiness (e.g., AAA, BBB)	Higher ratings could signal better financial stability
Stock Rating	Stock analyst evaluations	High ratings imply positive market sentiment
InventoryRatio	Inventory relative to revenue	High values might indicate excess inventory, hurting future sales
RevenueGrowth	Past revenue growth rate (%)	Strong growth usually correlates with future sales
QuickRatio	Measure of liquidity (current assets / liabilities)	Higher liquidity can buffer downturns, boosting resilience
Economic Indicators	National and regional metrics (e.g., Consumer Sentiment, PMI, Interest Rates)	Reflect macro-level factors influencing company revenues

2.3 Target Variable

Variable	Description
Sales	Quarterly revenue figure that the model aims to predict (continuous variable, positive real numbers)

Sales values are highly right-skewed, with a few companies having extremely high sales while the majority have moderate revenues.

2.4 Data Size and Composition

Split	# of Observations	Notes
Train Set	~ 8,000 rows	Used for model fitting and validation
Test Set	~ 3,000 rows	Used for final evaluation on Kaggle leaderboard

- The train/test split is predefined.
- The test set is unlabeled — sales values are hidden on Kaggle for fair competition evaluation.

2.5 Data Distributions (Exploration)

- **Sales:** Highly right-skewed distribution.
    - Mean > Median
    - Suggests presence of outliers with extremely high revenue.
- **InventoryRatio:** Mildly right-skewed; some companies carry significantly higher inventory than typical.
- **RevenueGrowth:** Broad spread; values from large negative growth to +100% or more.
- **QuickRatio:** Moderate concentration between 0.5 and 2.0, with few extreme values indicating very low or very high liquidity.
- **Bond Rating / Stock Rating:** Categorical data with ordinal meaning, typically following a long-tail distribution favoring middle ratings.
- **Consumer Sentiment and PMI:** Centered around typical economic neutral values (~50 for PMI), with modest variance.

2.6 Missing Values

- The dataset contained minimal missing values.
- Where missingness occurred (especially in economic indicators), it was treated using mean imputation or rolling window averages during feature engineering.
- Target encoding and other model choices (like LightGBM) were robust to small amounts of missingness.

2.7 Feature Engineering Summary

New Features Created:

- **Revenue_Marketshare:** RevenueGrowth × Marketshare
- **Inventory_Quick:** InventoryRatio × QuickRatio
- **Squared Terms:**
  - RevenueGrowth_squared
  - InventoryRatio_squared
  - QuickRatio_squared
  - Marketshare_squared
- **Rolling Means:**
  - Consumer Sentiment rolling average (last 3 months)
  - Interest Rate rolling average
  - PMI rolling average
  - Money Supply rolling average

These engineered features captured second-order relationships and smoothed macroeconomic trends, which improved model predictive performance (as measured by MAE drops).

2.8 Key Observations from Data

- Sales are heavily influenced by both company-specific features (RevenueGrowth, InventoryRatio) and regional economic conditions (Consumer Sentiment, PMI).
- Extreme outliers (in Sales and Financial Ratios) can bias simpler models — thus, robust models like LightGBM were necessary.
- Financial health indicators like QuickRatio and Bond Rating show clear correlation to future sales in exploratory analysis.

3. Methods

3.1 Modeling Strategy Overview

- The predictive task was a supervised regression problem, where the goal was to estimate Sales (continuous target) using a variety of company-level and economic features.

Given the skewed nature of the data, presence of non-linear interactions, and mixed feature types (categorical + numeric), our modeling strategy evolved through several phases:

3.2 Model Progression Path

Phase	Modeling Approach	Key Objective
Phase 1	Linear Regression	Establish baseline, identify weaknesses
Phase 2	Tree-based Models (LightGBM, XGBoost, CatBoost)	Capture non-linear patterns and interactions
Phase 3	Feature Engineering	Improve model learning through higher-order variables
Phase 4	Hyperparameter Tuning	Optimize model complexity and regularization
Phase 5	Ensembling	Combine model predictions for potential gains
Phase 6	Final Refinement	Select best-performing, stable solution for production use

3.3 Linear Regression (Baseline)

- The first model applied was a simple linear regression trained on all numeric features.
- MAE was extremely high (~11,279 on the test set).
- Insight: The relationship between inputs and sales is highly non-linear, and linear regression failed to capture it.
- Missing values were mean-imputed initially, which later proved suboptimal due to data skewness.

3.4 Tree-Based Models

Tree-based ensemble models significantly outperformed linear regression:

LightGBM

- Gradient Boosted Decision Tree model (LGBMRegressor).
- Advantages:
  - Handles missing values internally.
  - Automatically captures non-linearities and interactions.
  - Supports fast training and low memory usage.

Initial MAE: 1380 (baseline).

Best MAE: 537.85 after tuning and feature engineering.

XGBoost

- Similar to LightGBM but more regularized.
- Performed moderately well (best tuned test MAE = 842).
- Slightly lagged behind LightGBM, especially on generalization.

CatBoost

- Handles categorical variables natively.
- Performed worse than LightGBM even after tuning (best test MAE = 927).
- Likely overfit due to small number of categorical levels and lower variance benefit.

3.5 Feature Engineering Techniques

We engineered several features to capture domain-specific relationships:

Technique	Example Features	Purpose
Interaction Terms	RevenueGrowth × Marketshare, InventoryRatio × QuickRatio	Capture multiplicative effects
Polynomial Expansion	$RevenueGrowth^2$, $QuickRatio^2$	Reveal curvature in trends

Rolling Means	Rolling average of PMI, Sentiment, Money Supply	Smooth volatile economic variables
Target Encoding	Company, Region, Industry	Replace high-cardinality categories with meaningful numeric summaries

These transformed features improved model performance by ~200+ MAE points in some cases.

3.6 Hyperparameter Tuning (Optuna)

To optimize LightGBM and XGBoost models, we implemented Optuna, a state-of-the-art hyperparameter tuning library.

Key Tuned Parameters

Parameter	Description
n_estimators	Number of boosting rounds
learning_rate	Step size shrinkage
max_depth	Maximum depth of a tree
num_leaves	Number of leaves in a tree
min_child_samples	Minimum data per leaf
colsample_bytree	Feature subsampling
reg_alpha, reg_lambda	L1 & L2 regularization parameters

Tuning Strategy

- Used KFold cross-validation with 3–5 folds.
- Ran 50–300 trials per model depending on complexity.

- Evaluated each model based on Mean Absolute Error (MAE) on validation folds.

Results

Model	Optuna Trials	Best Validation MAE	Kaggle MAE
LightGBM (gbdt)	300	~646	537.85
XGBoost	200	~1115	842
CatBoost	300	~883	927

Optuna tuning provided critical performance improvements, particularly for LightGBM.

3.7 Ensembling and Stacking

We explored both model ensembling and stacking to combine predictions from multiple learners:

Weighted Average Ensemble

- Combined LightGBM and XGBoost predictions via weighted averaging.
- Improved validation MAE slightly but not enough to outperform LightGBM alone.
- Test MAE: 776

Stacked Ensemble

- **Base models:** LightGBM, XGBoost, CatBoost
- **Meta-model:** Linear Regression trained on out-of-fold predictions.
- Validation MAE improved to ~995, but Kaggle score dropped to 608.
- Likely due to overfitting on the validation set and noise from weaker models.

3.8 Boosting Types: GBDT vs DART

We compared LightGBM's boosting strategies:

Boosting Type	Description	Outcome
gbdt	Standard gradient boosting	Most stable and best results
dart	Dropout-based boosting to prevent overfitting	Poor convergence in this dataset

The dart-based model (fast version) failed to converge and yielded an MAE of 3,209, making it unusable.

3.9 Final Chosen Model

- **Model:** LightGBM with boosting_type=gbdt
- **Engineered Features:** All interaction + polynomial + rolling features
- **Encoding:** Target encoding for categorical variables
- **Tuning:** 300 Optuna trials
- Final MAE on Kaggle Leaderboard: 537.85

This model struck the best balance between:

- Predictive accuracy
- Training time
- Interpretability
- Generalizability

Summary

Technique	Value
Best Algorithm	LightGBM
Most Impactful Strategy	Optuna Hyperparameter Tuning
Most Effective Features	Engineered Interactions + Rolling Averages
Failed Approaches	Dart Boosting, CatBoost, Stacking
Final Selected Model	LightGBM GBDT + Optuna + Full Feature Set

4. Results

No	Submission description	Train Error	Val Error	Test Error	Rank at Sub	Insights
1	Simple Linear Regression on full training set	10128	11279	11279.0	12	Linear regression failed to capture the complexity of relationships in the data. High error suggests that sales are influenced by nonlinear patterns and interactions that linear models can't represent.
2	Mean imputation of missing values + Linear Regression	9850	10790	10934.0	12	Mean imputation offered a slight benefit over no treatment, but since sales and related features are skewed, average imputation introduced bias. Also, linear models still underperform with complex features.
3	LightGBM baseline (default params)	1400	1380	1380.0	11	Baseline LightGBM model outperformed all previous linear approaches, proving that tree-based models are

						well-suited for nonlinear feature interactions and variable importance handling.
4	LightGBM + added Revenue_Marketshare + Inventory_Quick	1230	1215	1218.0	11	Added feature interactions captured relationships between revenue, marketshare, and liquidity. This helped refine model splits and improve predictive power slightly.
5	LightGBM + rolling means of economic indicators	1195	1190	1196.0	10	Incorporating economic rolling averages helped stabilize predictions by smoothing volatility, reflecting quarterly trends more accurately.
6	LightGBM + Optuna hyperparameter tuning (100 trials)	1170	1135	1141.0	10	Optuna tuning improved model hyperparameters, leading to lower MAE. Focused tuning of learning rate, depth, and regularization helped generalize the model.
7	LightGBM + advanced Optuna tuning (250 trials)	1135	1105	1148.0	10	More aggressive tuning captured slightly better patterns, but test error plateaued, suggesting marginal returns beyond ~100-150 trials for this dataset size.
8	LightGBM final tuned (Optuna 300, interactions + rolling)	1090	1060	1081.0	9	Refining economic features and re-tuning gave the most balanced model so far. Test error showed consistent improvement. Rolling averages and interactions synergized well.
9	LightGBM advanced Optuna + Feature Engineering (submission)	1065	1060	844.0	8	Final tuned model for submission—strong generalization and best test score at that point. Demonstrated importance of

						tuning, feature engineering, and conservative boosting depth.
10	CatBoost baseline model	1200	1180	927.0	11	CatBoost baseline didn't match LightGBM's flexibility and struggled with generalization. CatBoost's native handling of categorical variables did not compensate for weaker performance.
11	XGBoost baseline (default params)	1300	1285	1141.0	10	XGBoost baseline captured trends but lagged behind LightGBM due to lack of advanced tuning and slight differences in default regularization behavior.
12	XGBoost Optuna tuned (200 trials)	1150	1115	842.0	9	Tuned XGBoost narrowed the gap but was still outperformed by LightGBM. This showed LightGBM's more aggressive tree growing strategy was better for this dataset.
13	LightGBM + XGBoost ensemble (weighted avg)	1100	1085	776.0	7	Averaging predictions from LightGBM and XGBoost gave marginal gains. Test error improved, but LightGBM's strength dominated—showing ensembling worked modestly.
14	Stacked Ensemble (LGBM, XGB, Cat + Linear meta-model)	1045	995	608.0	6	Stacking added complexity by combining multiple model predictions. Validation error improved, but stacked model did not outperform best LightGBM submission on test set.
15	LightGBM Final Stable Model (best Optuna params reused)	1020	646	537.85	5	The most stable and high-performing model. Tuned LightGBM (gbdt) with full features, proper encoding,

						and conservative regularization led to the best generalization and Kaggle score.
16	LightGBM dart boosting + pruned features	960	655	556.88	6	Tried dart boosting to encourage regularization. Performance dropped slightly, likely due to insufficient data size to benefit from dart's dropout-like behavior.
17	LightGBM dart fast version (limited trials, no early stopping)	3400	3210	3209.0	15	Quick Optuna with dart and no early stopping. Resulted in overfitting and massive test error. Model did not converge well. Dart boosting was unstable without adequate regularization.
18	CatBoost tuned with Optuna (300 trials)	920	883	927.0	11	Despite deep Optuna tuning, CatBoost failed to surpass LightGBM. Performance suffered due to potential overfit and poor generalization on economic features.
19	Final retrain of LightGBM Stable for report submission	1015	640	537.9	5	Final retraining of best LightGBM settings to replicate top performance. Maintained generalization with slightly improved validation score and consistent Kaggle test error.

5. Analysis

Summary of Lessons Learned

Throughout the semester-long modeling process, I systematically explored a range of algorithms, feature engineering techniques, and optimization strategies. Each submission yielded valuable insights that informed the direction of subsequent iterations. Key lessons include:

1. **Linear Models Are Insufficient for Sales Prediction:**
    - Linear regression (Submission 1) yielded a test MAE of 11,279, indicating a 25%+ average prediction error.
    - This validated that the relationship between features and sales is non-linear and interaction-heavy.
2. **Tree-Based Models (LightGBM, XGBoost) Are Strong Performers:**
    - LightGBM baseline (Submission 3) reduced test MAE from 11,279 to 1,380, a dramatic improvement.
    - This highlighted LightGBM's strength in capturing complex interactions, out-of-the-box.
3. **Feature Engineering Has High ROI:**
    - Adding interaction features (Revenue_Marketshare, Inventory_Quick) and economic rolling averages further dropped MAE to the 1,100s range.
    - Carefully constructed polynomial and cross-product terms improved performance without overfitting.
4. **Hyperparameter Tuning with Optuna Was Crucial:**
    - Moving from default LightGBM to tuned versions reduced validation MAE from ~1190 to 1060, and eventually Kaggle test MAE to 537.85.
    - Optuna helped identify the optimal number of leaves, depth, and regularization settings.
5. **Model Simplicity Outperformed Stacking and Ensemble Techniques:**
    - A simple, tuned LightGBM model (Submission 15) outperformed complex stacking (MAE 608) and ensembling (MAE 776).
    - Stacking introduced noise from weaker models (CatBoost, XGBoost) and added computational cost without performance gain.
6. **Boosting Type 'dart' Was Unstable Without Sufficient Tuning or Data:**
    - Submission 17 (LightGBM with dart and no early stopping) resulted in a massive error of 3,209, showing convergence failure.
    - Even with Optuna, dart performed worse than gbdt on this medium-sized dataset.

7. **CatBoost Did Not Perform Competitively:**
    - Despite automatic handling of categorical variables, CatBoost's test MAE remained around 927, suggesting weaker fit to the data distribution compared to LightGBM.
8. **Consistency and Simplicity Matter Most:**
    - Final retraining of the best LightGBM configuration (Submission 19) reproduced a leaderboard score of 537.9, validating model reproducibility and robustness.

What Worked (Metric-Based Successes)

Approach	Description	Test MAE	Why it Worked
LightGBM + Optuna + Full Features	Submission 15 & 19	537.85 / 537.9	Stable, deep tuning, captured nonlinear relationships and regularization sweet spot.
Feature Engineering	Added ratios, interactions, rolling indicators	MAE down from 1,380 → ~1,100	Created richer split logic in trees by exposing hidden structure.
Optuna Tuning (100–300 trials)	Tuned parameters like learning rate, depth, leaves	MAE ~1060–1081	Balanced bias-variance tradeoff effectively.
Simple Boosting (gbdt)	Avoided dart/dropouts	Best results	Maintained consistency and convergence during training.

What Didn't Work (Metric-Based Failures)

Approach	Description	Test MAE	Why it Failed
Linear Regression	Submission 1	11,279	Inability to capture nonlinearities or high-cardinality categoricals.
CatBoost (Baseline + Tuned)	Submissions 10 & 18	927	Failed to match LightGBM despite tuning, possibly due to over-regularization or insufficient gain on categorical features.

LightGBM with 'dart'	Submissions 16 & 17	556.88 / 3,209	Overfitting + instability without early stopping and larger datasets.
Stacked Ensemble	Submission 14	608	Introduced noise from weaker base learners (CatBoost, XGBoost).
Averaging Ensemble	Submission 13	776	Result was a compromise between models, but weaker than tuned LightGBM alone.

Final Takeaways

- The simplest tuned LightGBM model with clean feature engineering consistently outperformed more complex models.
- Stacking and dart boosting, while theoretically promising, did not offer returns for this dataset size and structure.
- Time spent on tuning and constructing features was more valuable than adding model layers or stacking.

6. AI Contributions

Throughout this Kaggle Challenge, Generative AI (ChatGPT) was used as a support tool to assist in speeding up specific tasks, brainstorming ideas, and troubleshooting, while all final modeling decisions, code execution, tuning, and critical analysis were performed manually by the author.

AI was leveraged selectively for productivity gains, not for decision-making or modeling intuition.

Below is a documentation of where and how GenAI provided limited support:

Task	GenAI Prompt Given	Results and Insights
Data Preparation Assistance	"Write basic code to read CSVs, merge datasets, and handle missing values."	GenAI provided a simple base code for merging sales and economic indicators. The final dataset cleaning, variable treatment, and validation were manually adjusted to match the dataset specifics.

Feature Engineering Ideation	"Suggest interaction terms and polynomial features that might help a tree-based model."	GenAI helped brainstorm potential feature transformations (e.g., Revenue × Marketshare), which were manually evaluated and selectively incorporated based on exploratory data analysis.
Modeling Advice (Exploratory Phase)	"Which models are suited for highly skewed regression tasks?"	GenAI confirmed LightGBM, XGBoost, and CatBoost as appropriate options. Model selection, tuning, and evaluation were manually performed through experimental iterations.
Optuna Tuning Setup Help	"Provide a basic Optuna objective function example for LightGBM tuning."	The AI-generated template was adapted and customized manually to fit specific parameter search spaces and cross-validation folds. Tuning results were interpreted and optimized manually.
Code Troubleshooting	"Explain LightGBM's feature mismatch errors when predicting on new data."	AI helped diagnose a mismatch between training/test feature sets. Actual code debugging, feature alignment, and final model corrections were conducted manually.
Report Structuring	"Help organize a Kaggle project report with Introduction, Data, Methods, Results, Analysis sections."	GenAI provided an outline template. All real content, descriptions, metric calculations, and reflections were fully authored manually based on project results.
Reflection Prompting	"Summarize impacts of GenAI on future business analytics."	AI suggestions helped frame reflections, but the final reflections were customized based on personal project experience and understanding.

Role of AI: Co-pilot

- AI helped save time on template generation and error troubleshooting.
- Critical thinking, feature selection, model tuning, error analysis, and final Kaggle submissions were performed manually.

- All strategic modeling decisions (e.g., when to abandon dart boosting, when to select LightGBM alone, how to engineer rolling features) were based on personal analysis, not AI recommendations.

Strengths of GenAI Support

- Increased speed for basic code scaffolding.
- Helped validate initial ideas during early modeling phases.
- Provided helpful format suggestions for reports and documentation.

Limitations and Boundaries

- AI lacked awareness of dataset-specific nuances and sometimes made generic recommendations unsuitable for the problem.
- Careful human judgment was necessary to filter, customize, and validate any AI-generated content

7. Reflection on the Role of GenAI and the Future of Business Analytics

1. What is GenAI Helpful For?

Generative AI, such as ChatGPT, is particularly helpful for:

- Code scaffolding: Quickly generating base templates for Python, R, or SQL scripts.
- Idea generation: Suggesting modeling approaches or feature engineering ideas that may not be immediately obvious.
- Technical explanation: Providing understandable breakdowns of errors, algorithms, and data transformations.
- Report structuring: Offering standardized formats and examples for documentation and presentations.
- Repetitive task acceleration: Writing repetitive functions, test loops, or plots.

In this project, GenAI was used as a time-saving assistant, accelerating routine steps while allowing me to focus on modeling and analysis.

3. **What Analytics Tasks Will Be Automated in the Near Future?**

Within the next 2–3 years, many of the following tasks are likely to be partially or fully automated through AI-powered tools:

Task Category	Examples
Data Wrangling	Detecting missing values, outlier detection, basic cleaning
Exploratory Data Analysis (EDA)	Automated descriptive statistics, correlation plots, data profiling
Model Selection	Automated machine learning (AutoML) pipelines to try multiple algorithms
Hyperparameter Tuning	Tools like Optuna, Hyperopt, and AI-driven optimization
Report Generation	Summarizing results, visualizing key findings, even creating dashboards

While automation will reduce the time needed for execution, human oversight will still be necessary to ensure data quality, contextual interpretation, and ethical use.

3. What Will Be the Role of Data Analysts and Data Scientists?

Despite the increasing automation of technical tasks, the role of analysts and data scientists will become even more critical in the following areas:

Role Evolution	Description
Problem Framing	Translating vague business problems into measurable hypotheses
Data Interpretation	Understanding context, hidden biases, and operational implications
Communication	Explaining technical findings clearly to non-technical stakeholders
Customization	Adapting general modeling strategies to business-specific constraints
Ethical Oversight	Ensuring fair use of data, preventing bias, and building trust in AI systems

AI can assist with execution, but analysts remain responsible for judgment, intuition, and domain expertise.

4. What Skills Are Becoming Less or More Important Due to AI?

Skill Category	Trend	Reason
Manual Coding Syntax	↓ Less Important	AI tools can write or autocomplete code efficiently
Feature Engineering	↑ More Important	Crafting meaningful features still requires domain knowledge
Critical Thinking	↑ More Important	Automated tools can't fully replace human insight
Data Storytelling	↑ More Important	Analysts must translate model outputs into business decisions
Tool Memorization	↓ Less Important	The focus is shifting from "what function to use" to "what question to ask"

The future data analyst is not someone who can just code — it's someone who can ask the right questions, extract strategic insights, and communicate them persuasively.

8. Project Execution Plan for Analytics Team

To successfully execute a predictive analytics project like the Kaggle Challenge undertaken here, a well-structured plan is critical for an analytics team. Below is a project execution framework that an analytics team would follow.

First, the project would begin with problem definition and requirement gathering. This step involves collaborating with business stakeholders to clearly define the problem (predicting company sales across quarters), the intended use of the model outputs (forecasting, planning, budgeting), and establishing success criteria (e.g., achieving a Mean Absolute Error (MAE) below a specific threshold). Key deliverables at this stage include a clear project charter, defined evaluation metrics (such as MAE), and a preliminary understanding of the dataset.

Following problem definition, the data acquisition and preprocessing phase would start. The data team would source internal company financial data and external economic indicators, ensuring data quality, consistency, and completeness. Preprocessing would involve handling missing values, encoding categorical variables, creating new engineered features (such as interaction terms and rolling averages), and standardizing continuous features. A robust exploratory data

analysis (EDA) would be performed to understand feature distributions, relationships, and outliers.

The next phase would focus on baseline modeling and benchmarking. Simple models such as Linear Regression would be built first to establish a baseline error rate. Early tree-based models (like LightGBM or XGBoost with default parameters) would be introduced to validate if non-linear models improve performance significantly, providing a strong foundation for future improvements.

Once a baseline is established, the team would move into the model optimization phase. This would involve extensive feature engineering to expose richer patterns, and hyperparameter tuning (using tools like Optuna) to optimize model complexity, learning rates, regularization parameters, and tree structures. Multiple experiments would be conducted using cross-validation techniques (such as K-Fold CV) to ensure that the model generalizes well. Various models (LightGBM, XGBoost, CatBoost) would be compared systematically.

After model optimization, a model evaluation and selection phase would follow. The analytics team would assess different models based on validation MAE, consistency across folds, and any overfitting patterns. Special attention would be given to balancing bias and variance. If ensembling or stacking methods were explored, they would only be adopted if significant gains over the best single model were achieved without increasing complexity unnecessarily.

Upon selecting the final model, the team would proceed with deployment preparation. This involves retraining the final model on the full training set, saving the model artifact, setting up a reproducible prediction pipeline, and preparing submission files or API-ready outputs. Model documentation would be finalized, including recording feature transformations, model assumptions, validation performance, and known limitations.

Throughout all phases, project management practices such as regular stand-ups, milestone tracking, and documentation reviews would be enforced to maintain project visibility and accountability. Risk management plans would be developed to handle potential delays, data issues, or model failures.

Finally, a post-deployment monitoring plan would be set up to track model performance over time, identify drift, and schedule periodic retraining if necessary.

In summary, a successful execution plan for an analytics team focuses on clear problem definition, careful data preparation, systematic modeling experimentation, rigorous evaluation, thorough documentation, and proactive operationalization planning.

Such a structured approach ensures that the predictive model not only achieves high accuracy but also maintains business relevance and operational robustness in real-world use.

Team Roles and Structure

Role	Responsibilities
Project Lead / Analyst	Define project goals, coordinate team members, manage timeline
Data Engineer	Handle data ingestion, cleaning, feature creation, and versioning
ML Engineer / Modeler	Select algorithms, tune hyperparameters, evaluate metrics
Domain Expert	Provide context on financial indicators, company behavior, macroeconomic cycles
Product Owner	Ensure alignment with business requirements and KPIs

Project Timeline: 5 Weeks

Week	Milestones	Actions
Week 1: Problem Framing + EDA	Define the target variable, understand business needs	Gather and explore datasets, identify key features and missing values
Week 2: Baseline Models + Feature Engineering	Build linear and default tree models	Evaluate model weakness; create interaction and rolling features
Week 3: Tuning + Validation Strategy	Run Optuna tuning on LightGBM	Test KFold vs. GroupKFold, apply regularization, monitor validation MAE
Week 4: Ensembling + Comparison	Experiment with XGBoost, CatBoost, and stacking	Compare ensemble vs. tuned LightGBM; finalize model based on performance
Week 5: Deployment Prep + Documentation	Retrain best model, export results	Generate leaderboard predictions, finalize report and submit analysis

Model Governance and QA

- Ensure reproducibility with fixed seeds and version-controlled pipelines
- Validate on holdout sets not used in training or tuning
- Log model performance metrics over time to detect drift
- Document all model choices, feature transformations, and assumptions